

Demand-Driven Gig Creation MVP

Stan Zvenigorodskiy

DevOps Lab Inc. - DevOpsLabInc.com

Security-remediated edition - July 31, 2026

Product Screenshots

- Landing page and campaign seed flow
- Campaign detail and supporter workflow
- Mobile campaign experience
- Project README
- Start here
- Current release summary
- Product idea
- Included

Facebook integration strategy

- The operating principle
- Why full Facebook Group syncing is not implemented

Recommended organizer workflow

- Phase 1 — Plant the seed
- Phase 2 — Connect each Facebook community
- Phase 3 — Share and pin
- Phase 4 — Measure demand
- Phase 5 — Reach the threshold
- Phase 6 — Confirm artist and venue
- Phase 7 — Convert to the normal VibesMeet event
- Phase 8 — Promote and measure attendance

Facebook integration matrix

Facebook App setup

1. Create a Meta app
2. Configure OAuth
3. Request Page permissions
4. Configure environment variables
5. Configure domains and privacy

Facebook API endpoints

- Read integration configuration
- Verify a Facebook Login access token
- List Pages managed by the organizer
- Generate a Group-specific share link
- Publish to a managed Facebook Page
- Send a server or offline conversion

Open Graph sharing

Meta Pixel and Conversions API

- Browser events
- Server events
- Data minimization

Campaign state machine

Run with Docker

Run without Docker

Core API examples

1. Plant the seed
2. Launch it
3. Support it from a Facebook community
4. Confirm artist and venue
5. Convert it into the confirmed event

Stripe configuration

Suggested VibesMeet integration boundary

Testing and validation

Production security and hardening

Important product distinction

Deep VibesMeet integration and missing-module blueprint

- Automated security testing
- Social authentication
- AWS production architecture

AWS Terraform framework

Product Screenshots

The following rendered interfaces show the demand-driven event workflow on desktop and mobile before the technical README explains the implementation.

Landing page and campaign seed flow

Open Concert x VibesMeet

Demand-driven events

DO NOT BOOK FIRST AND HOPE.

**Prove the audience.
Then make the gig happen.**

Plant a seed, gather real commitments, unlock sponsors, confirm the artist and venue, and convert verified demand into a live event.

Plant seed

 →

Gather fans

 →

Reach target

 →

Confirm venue

 →

Produce gig

Plant a gig seed

Campaign title

Bring Band X to New York

Artist or band

Band X

City

New York

Campaign deadline

August 28, 2026 at 8:00 PM

Why should this gig happen?

Commit now. The artist and venue are booked only when enough fans prove the demand.

Goal type

Both

Supporter target

500

Commitment target (\$)

25,000.00

Suggested deposit (\$)

25.00

Organizer name

Open Concert Community

Organizer email

organizer@openconcert.com

Plant the seed

Gig seeds

Campaigns become events only after demand is proven.

COLLECTING

Bring Band X to New York

New York 📅 Ends Aug 28, 2026

Commit now. The artist and venue are booked only when enough fans prove the demand.

Facebook Group

Rock Fans NYC

Referral tracking enabled

342

of 500 supporters

\$17,650

of \$25,000 committed

68%

minimum demand reached

Artist pending

✓ Venue option secured

✓ 3 sponsors interested

\$25.00

Make this gig happen

Support as a sponsor

126 supporters came from Facebook communities

View sources →

Deposits remain refundable until the campaign reaches its target and both the artist and venue are confirmed.

Demand Gig Engine - Stan Zvenigorodskiy - DevOpsLabInc.com - 4 / 26

Campaign detail and supporter workflow

Open Concert x VibesMeet

Share campaign ↗

Gig seeds / New York / Bring Band X

COLLECTING

Bring Band X to New York

A fan-powered concert campaign. No artist or venue is financially locked until verified demand reaches the published minimum.

New York, NY

Campaign closes August 28

Started by Open Concert Community

412

of 500 supporters

\$21,250

of \$25,000 committed

\$4,500

sponsor commitments

82%

minimum demand reached

88 more fans can unlock the booking process.

Once the threshold is reached, the organizer has 14 days to confirm the artist and venue. Deposits remain refundable until both confirmations are complete.

Make this gig happen

Your name

Email address

1 ticket commitment

\$25 refundable deposit

Support this gig with \$25

Your card is authorized now. The deposit becomes final only after the audience target, artist confirmation, and venue confirmation are complete.

Booking conditions

Audience threshold	82% reached
Artist availability	Awaiting threshold
Venue option	✓ Tentatively held
Sponsor coverage	✓ \$4,500 committed

Sponsor this audience

Local Sound Co.

Contact name

Business email

\$1,000.00

Logo placement and VIP activation area

Commit sponsorship

Audience sources

Facebook groups	154
Direct sharing	117
Creator profiles	89
Sponsor referrals	52

Top communities

Rock Fans NYC	86 fans
Live Music Brooklyn	41 fans
Band X Global	27 fans

Demand Gig Engine - Stan Zvenigorodskiy - DevOpsLabInc.com - 5 / 26

Mobile campaign experience

Open Concert x VibesMeet

Share campaign >

Gig seeds / New York / Bring Band X

COLLECTING

Bring Band X to New York

A fan-powered concert campaign. No artist or venue is financially locked until verified demand reaches the published minimum.

New York, NY

Campaign closes August 28

Open Concert Community

412
of 500 supporters

\$21,250
of \$25,000 committed

\$4,500
sponsor commitments

82%
minimum demand reached

88 more fans can unlock booking.
Deposits remain refundable until the artist and venue are both confirmed.

Make this gig happen

Your name

Email address

1 ticket commitment

\$25 refundable deposit

Support this gig with \$25

Your card is authorized now. The deposit becomes final only after every published condition is met.

Booking conditions

Audience threshold

82% reached

Artist availability

Awaiting threshold

Venue option

✓ Tentatively held

Sponsor coverage

✓ \$4,500 committed

Sponsor this audience

Local Sound Co.

Contact name

Business email

\$1,000.00

Logo placement and VIP activation area

Commit sponsorship

Audience sources

Facebook groups

154

Direct sharing

117

Creator profiles

69

Sponsor referrals

52

Top communities

Rock Fans NYC

88 fans

Live Music Brooklyn

41 fans

Band X Global

27 fans

Project README

Author: Stan Zvenigorodskiy
Organization: DevOps Lab Inc.
Website: DevOpsLabInc.com

A production-minded reference implementation of the missing Open Concert / VibesMeet workflow:

Plant the seed → gather supporters → reach the target → confirm the artist and venue → produce the gig.

This is not ordinary ticketing. A campaign proves real demand before an organizer assumes the full cost and risk of booking the artist, venue, production, security, insurance, travel, and promotion.

Start here

This repository is documented for three audiences:

1. **Product and business readers** can start with the demand-driven event flow and screenshots in this README.
2. **Application developers** should read [docs/CODE_WALKTHROUGH.md](#), then follow a request from `frontend/src/api.ts` to `backend/gigs/views.py`, `serializers.py`, `services.py`, and `models.py`.
3. **Cloud and DevSecOps engineers** should read [terraform/README.md](#) and [docs/terraform-module-architecture.md](#).

Documentation map

Document	Use it for
docs/CODE_WALKTHROUGH.md	File-by-file explanation of source, tests, workflows, and Terraform blocks
docs/DEVELOPER_ONBOARDING.md	Local setup, first change, debugging, testing, and pull-request workflow
README_FUTURE.md	Product roadmap and future-state architecture
docs/SOCIAL_AUTHENTICATION.md	Google, Facebook, Instagram, and TikTok authentication design
docs/VIBESMEET_INTEGRATION_AND_MODULE_BLUEPRINT.md	VibesMeet ownership boundaries, contracts, and integration modules
SECURITY_TESTING.md	Security gates, tools, CI jobs, and local commands
docs/CHECKOV_REMEDIATION.md	Checkov finding-by-finding remediation, exceptions, and verification
TEST_REPORT.md	Executed application validation and environment limitations
TERRAFORM_TEST_REPORT.md	Infrastructure test evidence and deployment-contract validation

Core business flow in plain English

A campaign begins as a proposal rather than a confirmed event. Fans and sponsors create verified demand. The backend counts only active commitments, checks the configured supporter and/or money threshold, moves the campaign through explicit states, confirms the artist and venue, and finalizes payments only when the business preconditions are satisfied. If the deadline passes without sufficient demand, refundable deposits are returned through the selected payment provider.

Author and stewardship

The architecture, implementation framework, and documentation are authored and maintained by **Stan Zvenigorodskiy**, **DevOps Lab Inc.** - DevOpsLabInc.com.

Current release summary

This package now combines the demand-driven gig workflow with production-oriented identity, authorization, AWS deployment, testing, and security controls:

- Social sign-in and account linking with Google, Facebook, Instagram, and TikTok through `django-allauth`.
- Gig profiles for fans, bands/artists, venues, organizers/promoters, equipment-rental companies, and sponsors.
- Authenticated campaign ownership; owner/staff authorization for launch, edit, confirmation, finalization, refund, deletion, conversion tracking, and Facebook Page publishing.
- Authenticated pledge and sponsor attribution while anonymous support remains available.
- CSRF-protected session APIs, a lightweight `/api/health/` endpoint, and provider credentials loaded only from environment variables.
- Production backend container using Gunicorn and WhiteNoise; Docker Compose passes OAuth configuration into the backend.
- AWS production blueprint covering CloudFront, WAF, ALB, ECS Fargate, RDS/RDS Proxy, Redis, SQS/DLQs, EventBridge, Secrets Manager, observability, backups, and multi-account governance.
- GitHub Actions for Python 3.10–3.12, frontend build, 90% coverage enforcement, CodeQL, Checkov, Bandit, dependency audits, dependency review, and an aggregate security gate.

The detailed implementation notes are in [SOCIAL_AUTH_AWS_CHANGELOG.md](#), [docs/SOCIAL_AUTHENTICATION.md](#), and [docs/AWS_PRODUCTION_ARCHITECTURE.md](#).

Product idea

A fan, artist, promoter, venue, sponsor, or community administrator proposes a gig such as:

Bring Band X to New York.

The organizer defines one or both minimum thresholds:

- 500 verified supporters
- \$25,000 in refundable commitments and sponsorships

Fans can register their intent, reserve a place, or pay a refundable deposit. Sponsors can commit after seeing verified audience demand. Only after the threshold is reached does the organizer confirm the artist and venue and convert the campaign into a normal ticketed VibesMeet event.

When the threshold is missed or the artist and venue cannot be confirmed under the published terms, deposits are refunded.

Included

Demand-campaign platform

- Django REST Framework API
- PostgreSQL-ready data model with SQLite fallback
- React + TypeScript + Vite interface
- Supporter attendance commitments
- Refundable supporter deposits
- Sponsor commitments
- Supporter-count, committed-money, or combined thresholds
- Atomic threshold evaluation to prevent race conditions
- Artist and venue confirmation gates
- Conversion into a live VibesMeet event through an `event_id`
- Failed-campaign refund workflow
- Stripe adapter and local fake-payment adapter
- Stripe webhook verification
- Idempotency keys for pledge submissions
- Immutable campaign-event audit log
- Automated tests for success and refund flows
- Docker Compose setup

Facebook-first organizer integration

- Existing Facebook Event URL attached to every gig seed
- Existing Facebook Group URL attached to every gig seed
- Artist, venue, or organizer Facebook Page URL
- Facebook Share Dialog with campaign-specific links
- Open Graph preview pages optimized for Facebook sharing
- Facebook Login with backend token validation
- Discovery of Facebook Pages managed by the connected organizer
- Publishing a campaign link to a managed Facebook Page
- Unique Group, community, and administrator referral codes
- Attribution of supporters, deposits, and revenue to each Facebook source
- Meta Pixel browser events
- Meta Conversions API server events
- Browser/server event IDs for deduplication
- Website, app, and physical attendance/check-in conversion events
- Facebook source data in the immutable campaign audit log

Facebook integration strategy

The operating principle

Event organizers already use **Facebook Events, Facebook Groups, Facebook Pages, Messenger, and community discussions** as their primary discovery and promotion channels.

The goal is not to force organizers to abandon those communities. The integration uses Facebook as the audience and communication layer while VibesMeet becomes the transaction, verification, sponsorship, and fulfillment layer.

```
Facebook Group / Event / Page
↓
Tracked "Make This Gig Happen" link
↓
VibesMeet demand campaign
↓
Verified supporter + refundable commitment
↓
Threshold reached
↓
Artist + venue confirmed
↓
Normal VibesMeet ticketed event
↓
Facebook Page/Event promotion + measured attendance
```

Why full Facebook Group syncing is not implemented

Meta retired the general-purpose Facebook Groups API and its permissions, including `publish_to_groups` and `groups_access_member_info`, for all versions on April 22, 2024.

Therefore, a compliant modern application cannot:

- Import a Facebook Group membership list
- Read private Group member data
- Automatically publish into arbitrary Groups
- Synchronize all Group posts and comments
- Automatically invite every Group member

This project implements the strongest supported alternative:

1. A Group administrator creates a unique tracked link.
2. The administrator manually posts or pins it in the Group or Facebook Event discussion.
3. Facebook generates a rich Open Graph preview.
4. Every visitor carries source, group, and ref attribution into VibesMeet.
5. VibesMeet measures supporters, deposits, sponsorships, conversions, and eventual attendance from that community.

Official Meta reference:

- <https://developers.facebook.com/docs/graph-api/changelog/version19.0/>

Recommended organizer workflow

Phase 1 — Plant the seed

The organizer creates the demand campaign and optionally attaches:

- Existing Facebook Event URL
- Main Facebook Group URL
- Artist, venue, or organizer Facebook Page URL
- Minimum supporter target
- Minimum financial target
- Suggested refundable deposit
- Campaign deadline

The Facebook Event can remain the familiar public discussion and discovery hub. The VibesMeet campaign becomes the authoritative place for verified demand.

Phase 2 — Connect each Facebook community

For each Facebook Group, Event discussion, Page, influencer, artist fan club, or promoter, generate a separate link:

```
https://your-domain.example/share/campaign/bring-band-x-new-york/  
?source=facebook_group  
&group=Band%20X%20NYC%20Fans  
&ref=admin-jane
```

Examples:

```
source=facebook_group&group=Band X NYC Fans&ref=admin-jane  
source=facebook_event&group=Official NYC Event&ref=event-discussion  
source=facebook_page&group=Band X Official&ref=page-organic  
source=facebook_ads&group=NYC Rock Audience&ref=adset-2026-07  
source=facebook_messenger&group=Street Team&ref=messenger-captain-4
```

This enables community-level reporting without copying Facebook member data.

Phase 3 — Share and pin

The organizer should place the tracked campaign URL in:

- Facebook Event ticket or external-link field where available
- Facebook Event description
- Event discussion pinned post
- Facebook Group pinned or featured post
- Facebook Page post
- Messenger community chats
- Artist and venue Page posts
- Relevant comments, with administrator approval

- Meta advertising campaigns

Suggested Facebook post:

Should Band X play New York?

The show is not booked yet. We are proving demand first.

Support the campaign, reserve your place, or leave a refundable deposit. Once the minimum audience and funding target is reached, the artist and venue will be confirmed and the gig will happen.

Make this gig happen:
<TRACKED_CAMPAIGN_LINK>

Phase 4 — Measure demand

Every pledge records:

- source
- source_label
- referral_code
- Supporter count
- Commitment amount
- Payment status
- Campaign and timestamp

Example pledge payload:

```
{
  "supporter_name": "Alex Fan",
  "supporter_email": "alex@example.com",
  "quantity": 1,
  "amount": "25.00",
  "idempotency_key": "browser-generated-uuid",
  "source": "facebook_group",
  "source_label": "Band X NYC Fans",
  "referral_code": "admin-jane"
}
```

This lets the organizer answer:

- Which Facebook Group produced the most supporters?
- Which Page generated the most deposits?
- Which administrator or promoter has the best conversion rate?
- Which geographic audience should receive the confirmed event announcement?
- Which community should receive a referral fee, recognition, or ticket allocation?

Phase 5 — Reach the threshold

When the configured minimum is reached, the campaign moves atomically from:

COLLECTING → TARGET_REACHED

The organizer then begins artist and venue confirmation. Supporter commitments are not silently treated as final ticket purchases before the published conditions are satisfied.

Phase 6 — Confirm artist and venue

The campaign records separate confirmation gates:

```
Artist confirmed: yes/no
Venue confirmed: yes/no
```

Both must be confirmed before the campaign becomes a live event.

Recommended production additions:

- Artist letter of intent
- Venue hold expiration date
- Capacity and accessibility
- Production rider
- Insurance
- Permit requirements
- Cancellation terms
- Deposit conversion terms

Phase 7 — Convert to the normal VibesMeet event

After artist and venue confirmation:

```
POST /api/campaigns/<slug>/finalize/
```

The Demand Campaign service publishes `campaign.finalized`. VibesMeet Event OS creates the ordinary confirmed event and returns its `event_id`.

At this point:

- Eligible deposits become final according to campaign terms
- Ticket inventory opens
- The event receives its standard ticketing page
- Facebook Event and Page posts can be updated with the confirmed date, venue, and ticket URL
- SponsorOS receives verified demand and source-attribution data

Phase 8 — Promote and measure attendance

Use both Meta Pixel and Conversions API to send funnel events such as:

```
PageView
ViewContent
Lead
InitiateCheckout
Purchase
```

AttendEvent	# custom physical check-in event
CampaignRefunded	# custom lifecycle event

For physical QR check-in, send a server event with:

```
{
  "event_name": "AttendEvent",
  "event_id": "checkin-vibes-event-123-ticket-456",
  "email": "fan@example.com",
  "action_source": "physical_store",
  "custom_data": {
    "event_id": "vibes-event-123",
    "ticket_id": "ticket-456",
    "venue": "Example Hall"
  }
}
```

API:

POST /api/campaigns/<slug>/facebook/track-conversion/

This gives organizers a complete measurement chain from Facebook discovery through real-world attendance.

Facebook integration matrix

Integration	Implemented	Notes
Existing Facebook Event URL	Yes	Stored with the campaign and displayed in the UI
Existing Facebook Group URL	Yes	Stored as the principal community hub
Facebook Page URL	Yes	Stored for the artist, venue, or organizer
Facebook Share Dialog	Yes	Opens a tracked campaign URL
Rich Facebook preview	Yes	Dynamic Open Graph page per campaign/source
Facebook Login	Yes	JavaScript SDK plus backend token validation
Managed Page discovery	Yes	Uses /me/accounts after permission approval
Publish campaign to Page	Yes	Uses /{page-id}/feed
Meta Pixel	Yes	Browser PageView and funnel events
Meta Conversions API	Yes	Server-side commitment, sponsor, purchase, and custom events
Browser/server deduplication	Yes	Shared event_id convention
Group member import	No	Groups API removed by Meta
Automatic arbitrary Group posting	No	Groups API removed by Meta
Automatic Group invitations	No	Not available through the current supported API
Existing Facebook Event creation/update	Manual workflow	Keep the Event as discovery hub and link it to the VibesMeet campaign
Messenger sharing	Link copy/share	Tracked URLs can be pasted into Messenger chats
Instagram promotion	Campaign URL	Use the same source/referral model for bio, stories, ads, and DMs
WhatsApp promotion	Campaign URL	Use tracked links for chats and communities
Offline attendance attribution	Yes	Custom Conversions API event with physical_store action source

Facebook App setup

1. Create a Meta app

Create a Meta developer application and add the relevant products:

- Facebook Login
- Marketing API / Conversions API as required by the selected business setup

Official documentation:

- <https://developers.facebook.com/docs/facebook-login/web/>
- <https://developers.facebook.com/documentation/ads-commerce/conversions-api>
- <https://developers.facebook.com/documentation/meta-pixel>

2. Configure OAuth

Add the production frontend domain and valid OAuth redirect URIs in the Meta application dashboard.

For local development, the frontend is normally:

```
http://localhost:5173
```

Production must use HTTPS.

3. Request Page permissions

The Page publishing workflow requests:

```
public_profile
email
pages_show_list
pages_read_engagement
pages_manage_posts
```

Depending on the app mode, organization, users, and intended public use, Meta may require App Review, a screencast, test credentials, business verification, and a clear explanation of how each permission is used.

The production application should request only the permissions actually needed.

Official Page publishing reference:

- <https://developers.facebook.com/docs/graph-api/reference/page/feed/>

4. Configure environment variables

Backend .env:

```
PUBLIC_BASE_URL=https://api.your-domain.example
META_GRAPH_API_VERSION=v25.0
META_APP_ID=your-meta-app-id
META_APP_SECRET=your-meta-app-secret
META_PIXEL_ID=your-pixel-id
META_CONVERSIONS_API_TOKEN=your-system-user-or-dataset-token
META_TEST_EVENT_CODE=
META_DEFAULT_SHARE_IMAGE=https://cdn.your-domain.example/default-gig-share.jpg
```

Frontend .env:

```
VITE_API_BASE=https://api.your-domain.example/api
VITE_STRIPE_PUBLISHABLE_KEY=pk_live_...
VITE_META_PIXEL_ID=
```

The frontend normally obtains the Pixel ID and Facebook App ID through:

```
GET /api/facebook/config/
```

The Meta App Secret and Conversions API token must never be exposed to the browser.

5. Configure domains and privacy

Before public release:

- Verify the production domain in Meta Business settings
- Publish privacy and data-deletion instructions
- Explain Facebook Login and advertising measurement in the privacy policy
- Obtain consent where required
- Respect Global Privacy Control and regional consent requirements
- Do not send prohibited or unnecessary personal data
- Hash permitted customer identifiers before sending server events
- Define retention and deletion schedules

Facebook API endpoints

Read integration configuration

```
curl http://localhost:8000/api/facebook/config/
```

Verify a Facebook Login access token

```
curl -X POST http://localhost:8000/api/facebook/login/ \
-H 'Content-Type: application/json' \
-d '{"access_token":"USER_ACCESS_TOKEN"}
```

The backend validates the token against the configured Meta App and then requests the connected user profile.

List Pages managed by the organizer

```
curl -X POST http://localhost:8000/api/facebook/pages/ \
-H 'Content-Type: application/json' \
-d '{"access_token":"USER_ACCESS_TOKEN"}'
```

Generate a Group-specific share link

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/facebook/share-link/ \
-H 'Content-Type: application/json' \
-d '{
  "source":"facebook_group",
  "group_name":"Band X NYC Fans",
  "referral_code":"admin-jane"
}'
```

Response:

```
{
  "campaign_url": "https://api.example.com/share/campaign/bring-band-x-new-york/?
    source=facebook_group&group=Band+X+NYC+Fans&ref=admin-jane",
  "share_dialog_url": "https://www.facebook.com/dialog/share?..."
}
```

Publish to a managed Facebook Page

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/facebook/publish-page/ \
-H 'Content-Type: application/json' \
-d '{
  "page_id":"FACEBOOK_PAGE_ID",
  "page_access_token":"PAGE_ACCESS_TOKEN",
  "source":"facebook_page",
  "referral_code":"official-page",
  "message":"Should Band X play New York? Support the seed and help make the gig happen."
}'
```

The access token is used for the request and is not written into the campaign audit event.

Send a server or offline conversion

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/facebook/track-
  conversion/ \
-H 'Content-Type: application/json' \
-d '{
  "event_name":"AttendEvent",
  "event_id":"checkin-ticket-456",
  "email":"fan@example.com",
  "action_source":"physical_store",
}
```

```
"currency": "USD",
"custom_data": {
  "event_id": "vibes-event-123",
  "ticket_id": "ticket-456"
}
```

Open Graph sharing

Facebook should share the backend URL rather than the JavaScript-only frontend URL:

```
/share/campaign/<slug>/
```

That page contains server-rendered metadata:

```
<meta property="og:type" content="website">
<meta property="og:title" content="Bring Band X to New York">
<meta property="og:description" content="500 supporters needed...">
<meta property="og:url" content="https://.../share/campaign/bring-band-x-new-york/">
<meta property="og:image" content="https://cdn.../campaign-image.jpg">
```

It also displays a lightweight campaign landing page and routes supporters to the React interface while preserving source, Group, and referral parameters.

Official sharing documentation:

- <https://developers.facebook.com/documentation/sharing/web>
- <https://developers.facebook.com/documentation/sharing/reference/share-dialog/>

Meta Pixel and Conversions API

Browser events

The frontend loads Meta Pixel only when a Pixel ID is configured.

A commitment uses the returned pledge ID as the browser event ID:

```
pledge:<pledge-uuid>:created
```

Server events

The backend sends the same event ID to Conversions API, allowing Meta to deduplicate browser and server versions of the event.

Examples:

Lead	attendance commitment without deposit
InitiateCheckout	refundable deposit initiated/recorded
Purchase	deposit finalized after gig confirmation
AttendEvent	physical QR check-in

The core campaign flow never depends on Meta attribution availability. A Meta API outage must not prevent a supporter from committing, paying, receiving a refund, or entering the event.

Data minimization

The reference implementation hashes email addresses with SHA-256 before sending them to Conversions API. A production system should additionally handle consent, cookies, _fbp, _fbc, client IP, user agent, regional restrictions, and data-processing terms in accordance with applicable law and Meta requirements.

Campaign state machine

```

DRAFT
  ↓ launch
COLLECTING
  ↓ target reached
TARGET_REACHED
  ↓ artist or venue confirmation begins
CONFIRMING
  ↓ both confirmed
CONFIRMED
  ↓ deposits finalized and event created
LIVE
  ↓ event occurs
COMPLETED

COLLECTING + deadline missed → REFUNDING → REFUNDED
Any pre-live cancellation    → REFUNDING → REFUNDED

```

Run with Docker

```

cd demand-gig-mvp
docker compose up --build

```

Open:

- Frontend: <http://localhost:5173>
- API: <http://localhost:8000/api/campaigns/>
- Admin: <http://localhost:8000/admin/>

Create a demo campaign and an admin account:

```

docker compose exec backend python manage.py seed_demo
docker compose exec backend python manage.py createsuperuser

```

Apply the Facebook URL migration:

```
docker compose exec backend python manage.py migrate
```

Run without Docker

Backend:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python manage.py migrate
python manage.py runserver
```

Frontend:

```
cd frontend
npm install
npm run dev
```

Core API examples

1. Plant the seed

```
curl -X POST http://localhost:8000/api/campaigns/ \
-H 'Content-Type: application/json' \
-d '{
  "title": "Bring Band X to New York",
  "pitch": "Prove demand before booking the artist and venue.",
  "artist_name": "Band X",
  "city": "New York",
  "country": "United States",
  "deadline": "2026-09-01T23:59:00-04:00",
  "goal_type": "both",
  "supporter_target": 500,
  "amount_target": "25000.00",
  "suggested_deposit": "25.00",
  "currency": "USD",
  "organizer_name": "Open Concert Community",
  "organizer_email": "organizer@example.com",
  "facebook_event_url": "https://www.facebook.com/events/example",
  "facebook_group_url": "https://www.facebook.com/groups/example",
  "facebook_page_url": "https://www.facebook.com/example"
}'
```

2. Launch it

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/launch/ \
-H 'Content-Type: application/json' -d '{}'
```

3. Support it from a Facebook community

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/pledge/ \
-H 'Content-Type: application/json' \
-d '{
  "supporter_name": "Alex Fan",
  "supporter_email": "alex@example.com",
  "quantity": 1,
  "amount": "25.00",
  "idempotency_key": "browser-generated-uuid",
  "source": "facebook_group",
  "source_label": "Band X NYC Fans",
  "referral_code": "admin-jane"
}'
```

4. Confirm artist and venue

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/confirm-artist/ \
-H 'Content-Type: application/json' -d '{"details": "Signed artist LOI dated August 20"}'

curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/confirm-venue/ \
-H 'Content-Type: application/json' -d '{"details": "Venue hold and capacity approved"}'
```

5. Convert it into the confirmed event

```
curl -X POST http://localhost:8000/api/campaigns/bring-band-x-to-new-york/finalize/ \
-H 'Content-Type: application/json' -d '{"event_id": "vibesmeet-event-123"}'
```

Stripe configuration

Backend:

```
PAYMENT_PROVIDER=stripe
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
```

Frontend:

```
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_...
```

Webhook:

```
/api/payments/stripe/webhook/
```

The implementation charges a refundable deposit rather than calling it escrow. Long campaign periods generally should not depend on card authorization holds. If the target is missed, a refund is issued to the original `PaymentIntent`.

Suggested VibesMeet integration boundary

Keep demand campaigns as their own bounded context and publish events when state changes occur:

```
campaign.launched
facebook.share_link_created
facebook.page_post_published
facebook.conversion_forwarded
pledge.created
sponsor.pledged
campaign.target_reached
artist.confirmed
venue.confirmed
campaign.finalized
pledge.refunded
```

VibesMeet Event OS subscribes to `campaign.finalized`, creates the normal ticketed event, and returns the event ID.

SponsorOS consumes:

- Supporter geography
- Facebook source and community attribution
- Verified supporter total
- Verified committed amount
- Sponsor commitments
- Conversion rate by Group/Page/referral code
- Confirmed ticket sales
- QR attendance
- Sponsor impressions and post-event reporting

Testing and validation

Run the complete application suite

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
./scripts/run_full_tests.sh
```

This runs dependency-free structural checks, GitHub workflow validation, shell validation, Django configuration and migration checks, blocking Flake8 rules, pytest with line/branch coverage, the frontend TypeScript/Vite build, and Docker Compose configuration validation when Docker is available.

Run application and security tests

```
./scripts/run_all_tests.sh
```

The security phase runs Checkov, Bandit, pip-audit, npm audit, and workflow validation. CodeQL and pull-request dependency review run in GitHub Actions because they depend on GitHub's code-scanning environment.

Run individual checks

```
python scripts/static_checks.py
python scripts/validate_workflows.py
python -m pytest backend -v
./scripts/security_scan.sh
```

Backend CI requires at least **90% line and branch coverage** for the business-logic scope defined in [COVERAGE_POLICY.md](#). The current package contains 74 Python test functions, including expanded social-auth, ownership, profile, health-check, pledge, and sponsor attribution tests.

The previous GitHub baseline completed 84 tests plus 7 subtests with 99.37% coverage before the final social-auth expansion. The new social-auth package must be pushed as a new commit so GitHub can perform the definitive Django/Allauth, frontend, and security execution. This sandbox could not download Django or npm dependencies, so blocked runtime checks are reported honestly in [TEST_REPORT.md](#) rather than counted as passes.

The combined README and product screenshots remain available at [docs/Demand_Driven_Gig_MVP_README_and_Screenshots.pdf](#).

Production security and hardening

Implemented controls include session authentication, OAuth state handling through Allauth, CSRF-protected mutations, environment-only provider secrets, campaign owner/staff authorization, secure-cookie/HSTS production defaults, Gunicorn, non-root containers, Checkov, Bandit, CodeQL, dependency auditing, workflow validation, and a required security gate.

Remaining production work:

1. Add organization membership and granular RBAC beyond campaign owner/staff checks.
2. Require recent reauthentication for account linking, settlement changes, refunds, and destructive actions.
3. Add rate limiting and bot protection for `/accounts/*`, `/api/auth/*`, pledges, sponsors, and publishing endpoints.
4. Store long-lived publishing tokens only in encrypted, scoped connection records; login tokens remain unstored by default.
5. Run payment, refund, notification, provider-webhook, and Meta conversion work asynchronously through SQS with retries and DLQs.
6. Add durable idempotency for external events, a double-entry payout ledger, and reconciliation reporting.
7. Add consent management, privacy/deletion workflows, retention schedules, and regional data-processing controls.
8. Add organization verification, sponsor due diligence, fraud/velocity controls, and sanctions screening where required.
9. Add contract, insurance, permit, production-rider, capacity, accessibility, cancellation, and chargeback workflows.
10. Complete provider review and staging smoke tests for each Google, Meta/Instagram, and TikTok application before enabling it in production.

11. Protect production with WAF managed/rate rules, GuardDuty, Security Hub, Inspector, Config, CloudTrail, centralized logs, and tested backups.
 12. Obtain legal review of refundable commitment, deposit conversion, tax, payout, privacy, and event-cancellation terms.
-

Important product distinction

Ordinary event platforms work like this:

```
Book artist and venue → spend money → publish tickets → hope enough people attend
```

This product works like this:

```
Propose the gig → prove the audience → verify money and sponsors → confirm artist and venue → produce the event
```

That is the core Open Concert concept and the missing demand-validation layer that can sit in front of VibesMeet Event OS and SponsorOS.

Deep VibesMeet integration and missing-module blueprint

The expanded product and integration design is documented in:

- [docs/VIBESMEET_INTEGRATION_AND_MODULE_BLUEPRINT.md](#)
- [docs/openapi/vibesmeet-bridge.openapi.yaml](#)
- [docs/schemas/vibesmeet-event-handoff.schema.json](#)
- [docs/schemas/vibesmeet-webhook-envelope.schema.json](#)
- [backend/integrations/vibesmeet/](#)

The proposed boundary is:

```
Demand Gig Engine owns demand validation, feasibility, artist/venue assembly, readiness, and pre-confirmation reservations.
```

```
VibesMeet owns confirmed-event ticketing, checkout, secure QR access, payouts, verified attendance, and post-event reporting.
```

The integration scaffold is deliberately contract-first. It does not assume that the proposed private VibesMeet endpoint names are already available. Authentication, scopes, payloads, rate limits, webhooks, and reservation-credit behavior must be confirmed with VibesMeet before production use.

Coverage quality gate

Backend CI enforces **90% minimum line and branch coverage for production business logic**. The exact measured modules and the narrowly defined framework/bootstrap exclusions are documented in [COVERAGE_POLICY.md](#).

```
python -m pytest backend -v
```

The command fails automatically when measured coverage is below 90% and writes `coverage.xml` for GitHub Actions.

Automated security testing

GitHub Actions runs Checkov, Bandit, pip-audit, npm audit, CodeQL, dependency review, and workflow validation. The aggregate **Security gate** can be required in branch protection. See [SECURITY_TESTING.md](#) for scanner scope, enforcement levels, reports, and repository settings.

Social authentication

The gig module now supports Google, Facebook, Instagram, and TikTok login through `django-allauth`. Social identities map to one local profile with fan, band, venue, organizer, rental, or sponsor account types. Authenticated users can link additional providers, campaign creation requires login, and lifecycle actions are restricted to the campaign owner or staff. See [docs/SOCIAL_AUTHENTICATION.md](#) for provider callbacks, environment variables, production setup, and the `social-auth` flow diagram.

AWS production architecture

The recommended production topology is documented in [docs/AWS_PRODUCTION_ARCHITECTURE.md](#), with a rendered diagram in [docs/aws-production-architecture.svg](#).

AWS Terraform framework

A complete modular AWS framework is available under [terraform/](#). It implements 25 reusable modules and isolated development/production stacks with native S3 state lockfiles, CloudFront/WAF/S3, a CloudFront-only ALB origin, ECS Fargate API/worker/migration tasks, PostgreSQL with RDS Proxy, Redis, SQS/EventBridge, SES, Secrets Manager/KMS, CloudWatch, CloudTrail, GuardDuty, X-Ray, AWS Backup, ECR, and GitHub OIDC.

The deployment workflow builds and pushes both images, provisions services at zero capacity, injects optional provider credentials, runs a dedicated migration task, scales API/worker services, uploads the React build with safe cache headers, and invalidates CloudFront. The browser uses same-origin `/api` in AWS; Docker Compose injects the local backend URL during the frontend build.

```
./terraform/scripts/validate.sh
cd terraform/tests && go test -race -count=1 -v ./...

# Deploy after AWS authentication
```

```
PROVIDER_CREDENTIALS_FILE=/secure/provider-credentials.json \  
./terraform/scripts/deploy.sh dev  
./terraform/scripts/deploy.sh prod
```

Architecture and module mapping: [docs/terraform-module-architecture.md](#).

Executed/deferred test matrix: [TERRAFORM_TEST_REPORT.md](#).